Documentos: MongoDB

Mikel Egaña Aranguren

mikel-egana-aranguren.github.io

mikel.egjavi@ehu.eus

Universidad del País Vasco
Euskal Herriko Unibertsitatea
BILBOKO INGENIARITZA ESKOLA
ESCUELA DE INGENIERÍA DE BILBAO

Documentos: MongoDB

https://github.com/mikel-egana-aranguren/ABD

CC BY

MongoDB

Base de datos NoSQL orientada a **documentos**: Los datos se almacenan como documentos JSON

No requiere un esquema fijo (schemaless)

Herramientas

- **mongod** — el servidor de base de datos
- **mongosh** — shell interactiva para ejecutar comandos
- **MongoDB Compass** — interfaz gráfica para explorar colecciones
- **MongoDB Atlas** — servicio cloud gestionado

Modelo de datos

Concepto	MongoDB
Base de datos	Base de datos
Unidad de organización	Colección
Unidad de dato	Documento
Atributo	Campo (field)
Identificador	_id
Relaciones	Relaciones entre documento: embebidos / referencias

JSON: JavaScript Object Notation

Formato de intercambio de datos ligero, legible por humanos y máquinas

JSON (ECMA 404)

Basado en pares **clave: valor**

Tipos de datos: strings, números, booleanos, arrays, objetos, null

Los objetos pueden contener otros objetos y arrays de objetos: modelar datos jerárquicos de forma directa

Documentos JSON

```
{
  "nombre": "Javier Cansado",
  "direccion": {
    "calle": "Gran Vía 1",
    "ciudad": "Bilbao",
    "cp": "48001"
  },
  "telefonos": [
    {"tipo": "movil", "numero": "666111222"},
    {"tipo": "casa", "numero": "944123456"}
  ],
  "viviendas": ["Deusto", "Abando", "Begoña"]
}
```

BSON: Binary JSON

MongoDB almacena los documentos internamente en formato **BSON**

Representación binaria de JSON, más eficiente para almacenamiento

BSON: Binary JSON

Soporta tipos adicionales que JSON no tiene:

- `ObjectId` — identificador único de 12 bytes
- `Date` — fechas con milisegundos
- `NumberInt`, `NumberLong`, `NumberDecimal`
- `BinData` — datos binarios
- `Regex` — expresiones regulares

ObjectId

Cada documento tiene un campo `_id` que actúa como clave primaria

Si no se proporciona, MongoDB genera un `ObjectId` automáticamente

Es único globalmente, ordenable por tiempo de creación

Se genera de forma automática y funciona bien en sistemas distribuidos

Conectar y crear una base de datos

```
// Conectar al servidor (Terminal o GUI)
$ mongosh
$ mongodb-compass

// En MongoDB basta con usar la BD (se crea al insertar)
test> use ehu

// Ver las colecciones de la BD actual
ehu> show collections
```

MongoDB crea las bases de datos y colecciones **de forma implícita** al insertar el primer documento

CRUD: Operaciones básicas

Operación	Métodos MongoDB
Crear	insertOne() / insertMany()
Leer	find() / findOne()
Actualizar	updateOne() / updateMany()
Borrar	deleteOne() / deleteMany()

insertOne()

```
db.alumni.insertOne({
  nombre: "Javier Cansado",
  email: "javi@ehu.eus",
  edad: 21
})
```

No hace falta definir la tabla colección previamente

No hace falta que todos los documentos tengan los mismos campos

MongoDB inserta un ID automáticamente

db.alumni.findOne({ _id: ObjectId("XXX") }) / db.alumni.find()

insertMany()

```
db.alumni.insertMany([
  {
    nombre: "Jon Aguirre",
    email: "jon@ehu.eus",
    edad: 22,
    notas: [
      { asignatura: "ABD", nota: 7.5 }
    ]
  },
  {
    nombre: "Miren Etxebarria",
    email: "miren@ehu.eus",
    edad: 20
  },
  {
    nombre: "Unai López",
    email: "unai@ehu.eus",
    edad: 23,
    becado: true
  }
])
```

Schemaless: flexibilidad

Cada documento puede tener campos diferentes

Todos conviven en la misma colección sin problemas

find(): consultar documentos

```
db.alumni.find()

db.alumni.find({ edad: 20 })

db.alumni.find({ nombre: "Jon Aguirre" })

// findOne() devuelve solo el primer resultado
```

Operadores de comparación

```
db.alumni.find({ edad: { $gt: 21 } })

db.alumni.find({ edad: { $gte: 20, $lte: 23 } })

db.alumni.find({ edad: { $ne: 21 } })

db.alumni.find({ edad: { $in: [20, 21, 22] } })
```

`$gt`, `$gte`, `$lt`, `$lte`, `$eq`, `$ne`, `$in`, `$nin`

Operadores lógicos

```
db.alumni.find({
  $and: [
    { edad: { $gt: 20 } },
    { nombre: "Javier Cansado" }
  ]
})

db.alumni.find({
  $or: [
    { edad: { $lt: 22 } },
    { becado: true }
  ]
})
```

`$and`, `$or`, `$not`, `$nor`

AND implícito

Cuando hay múltiples campos, MongoDB aplica AND:

```
db.alumni.find({ edad: 21, nombre: "Javier Cansado" })


db.alumni.find({
  $and: [
    { edad: 21 },
    { nombre: "Javier Cansado" }
  ]
})
```

Proyección (elegir campos)

```
db.alumni.find(
  {}, // filtro: todos
  { nombre: 1, email: 1 } // proyección: solo estos campos
)

// Excluir campos (mostrar todos menos email):
db.alumni.find(
  {},
  { email: 0 }
)

// El campo _id se incluye siempre por defecto
// Para excluirlo:
db.alumni.find({}, { nombre: 1, _id: 0 })
```

1 = incluir, 0 = excluir. No se pueden mezclar (excepto `_id`)

Ordenar, limitar

```
db.alumni.find().sort({ edad: 1 })

db.alumni.find().sort({ edad: -1 })

db.alumni.find().limit(3)

db.alumni.find().sort({ edad: -1 }).limit(2)
```

Contar documentos

```
db.alumni.countDocuments()

db.alumni.countDocuments({ edad: { $gt: 21 } })

// Contar con estimación rápida (para colecciones grandes):
db.alumni.estimatedDocumentCount()
```

updateOne()

```
db.alumni.updateOne(
  { nombre: "Javier Cansado" }, // filtro
  { $set: { edad: 22 } } // actualización (UPSERT)
)

// $set modifica solo los campos indicados
// Si el campo no existe, lo crea:
db.alumni.updateOne(
  { nombre: "Javier Cansado" },
  { $set: { telefono: "666111222" } }
)
```

Operadores de actualización

```
// Incrementar un valor numérico
db.alumni.updateOne(
  { nombre: "Jon Aguirre" },
  { $inc: { edad: 1 } }
)

// Eliminar un campo del documento
db.alumni.updateOne(
  { nombre: "Unai López" },
  { $unset: { becado: "" } }
)

// Renombrar un campo
db.alumni.updateOne(
  { nombre: "Javier Cansado" },
  { $rename: { "telefono": "tlfno" } }
)
```

`$set`, `$unset`, `$inc`, `$rename`, `$min`, `$max`, `$mul`

updateMany()

```
db.alumni.updateMany(
  { edad: { $gte: 20 } },
  { $set: { activo: true } }
)

// Resultado:
// { acknowledged: true,
// matchedCount: 4,
// modifiedCount: 4 }
```

`updateOne` modifica solo el primer documento que coincida

`updateMany` modifica todos los documentos que coincidan

Operaciones sobre arrays

```
// Añadir un elemento a un array
db.alumni.updateOne(
  { nombre: "Jon Aguirre" },
  { $push: { notas: { asignatura: "SI", nota: 8.0 } } }
)

// Añadir varios elementos a un array
db.alumni.updateOne(
  { nombre: "Jon Aguirre" },
  { $push: { notas: { $each: [
    { asignatura: "ED", nota: 6.5 },
    { asignatura: "SO", nota: 7.0 }
  ]}}}
)

// Eliminar un elemento de un array
db.alumni.updateOne(
  { nombre: "Jon Aguirre" },
  { $pull: { notas: { asignatura: "ED" } } }
)
```

`$push`, `$pull`, `$addToSet`, `$pop`, `$each`

deleteOne() y deleteMany()

```
db.alumni.deleteOne({ nombre: "Unai López" })

db.alumni.deleteMany({ edad: { $lt: 20 } })

// Borrar todos los documentos de la colección
db.alumni.deleteMany({})

// Eliminar la colección completa
db.alumni.drop()

// Eliminar la base de datos actual
db.dropDatabase()
```

replaceOne()

```
// Reemplaza un documento ENTERO (no campos individuales)
db.alumni.replaceOne(
  { nombre: "Javier Cansado" },
  {
    nombre: "Javier Cansado Fernández",
    email: "javier.cansado.fernandez@ehu.eus",
    edad: 22,
    grado: "Ingeniería Informática"
  }
)
```

Cuidado: `replaceOne()` sustituye todo el documento

A diferencia de `updateOne()` con `$set`, que modifica solo campos específicos

Consultas en documentos anidados

```
Datos: mongodb-compass; add data; insert document (nested.json)

// Buscar por campo anidado (dot notation):
db.alumni.find({ "direccion.ciudad": "Bilbao" }) // mongosh
{ "direccion.ciudad": "Bilbao" } // mongodb-compass
```

Dot notation: `"campo.subcampo"` para acceder a campos anidados

Consultas en arrays

```
Datos: nested-arrays.json (mongodb-compass: import)

// Buscar documentos que contengan "euskera" en el array:
db.alumni.find({ idiomas: "euskera" })
// Resultado: Jon

// Buscar documentos con el array exacto:
db.alumni.find({ idiomas: ["castellano", "francés"] })
// Resultado: Mikel (orden importa)

// Buscar por tamaño del array:
db.alumni.find({ idiomas: { $size: 3 } })
// Resultado: Jon (tiene 3 idiomas)
```

Consultas en arrays de objetos

```
{
  nombre: "Jon",
  notas: [
    { asignatura: "ABD", nota: 7.5 },
    { asignatura: "SI", nota: 8.0 }
    { asignatura: "SO", nota: 7.0 }
  ]
}

// Buscar ehu con nota > 7 en alguna asignatura:
db.alumni.find({ "notas.nota": { $gt: 7 } })

// Buscar ehu con nota > 7 en ABD específicamente:
db.alumni.find({
  notas: {
    $elemMatch: {
      asignatura: "ABD",
      nota: { $gt: 7 }
    }
  }
})
```

`$elemMatch` : el mismo elemento debe cumplir todas las condiciones

$exists y $type

```
// Buscar documentos que TENGAN el campo "becado":
db.alumni.find({ becado: { $exists: true } })

// Buscar documentos que NO tengan el campo "telefono":
db.alumni.find({ telefono: { $exists: false } })

// Buscar documentos donde "edad" sea de tipo numérico:
db.alumni.find({ edad: { $type: "number" } })

// Buscar documentos donde "nombre" sea un string:
db.alumni.find({ nombre: { $type: "string" } })
```

Muy útiles en colecciones schemaless donde no todos los documentos tienen los mismos campos

Expresiones regulares

```
db.alumni.find({ nombre: /^Mi*/ })

db.alumni.find({ email: /ehu\.eus$/ })

// Búsqueda case-insensitive:
db.alumni.find({ nombre: { $regex: "Cansado", $options: "i" } })
```

Expresiones regulares PCRE (Perl Compatible Regular Expressions)

Aggregation Pipeline

Sistema de procesamiento de datos en etapas (stages)

Los documentos pasan por una cadena de transformaciones

Permite filtrar, transformar, agrupar y enriquecer datos paso a paso

```
db.coleccion.aggregate([ { $etapa1: { ... } }, { $etapa2: { ... } }, { $etapa3: { ... } } ])
```

La salida de una etapa es la entrada de la siguiente

Etapas principales del Pipeline

Etapa	Descripción
`$match`	Filtra documentos
`$group`	Agrupa y calcula
`$project`	Selecciona/transforma campos
`$sort`	Ordena resultados
`$limit`	Limita resultados
`$unwind`	Descompone arrays
`$lookup`	Une colecciones

$match: filtrar

```
db.alumni.aggregate([
  { $match: { edad: { $gt: 20 } } }
])

// La sintaxis del filtro es la misma que en find()
// $match debería ir lo antes posible para reducir
// el número de documentos que pasan a las siguientes etapas
```

$group: agrupar y calcular

```
Nueva colección "pedidos", cargar pedidos.json

db.pedidos.aggregate([
  {
    $group: {
      _id: "$cliente",
      total: { $sum: "$precio" },
      num_pedidos: { $sum: 1 }
    }
  }
])
```

Acumuladores de $group

MongoDB	Descripción
`$sum`	Suma valores o cuenta
`$avg`	Media aritmética
`$min`	Valor mínimo
`$max`	Valor máximo
`$first`	Primer valor del grupo
`$push`	Array con todos los valores

Pipeline completo: ejemplo

```
db.pedidos.aggregate([
  { $match: { precio: { $gt: 30 } } },            // WHERE
  { $group: {                              // GROUP BY
      _id: "$cliente",
      total: { $sum: "$precio" }
  }},
  { $match: { total: { $gt: 200 } } },        // HAVING
  { $sort: { total: -1 } }                    // ORDER BY
])
```

El pipeline permite construir consultas complejas paso a paso

$project: transformar campos

```
db.alumni.aggregate([
  {
    $project: {
      nombre: 1,
      email_mayus: { $toUpper: "$email" },
      anio_nacimiento: {
        $subtract: [2026, "$edad"]
      },
      _id: 0
    }
  }
])
```

$unwind: descomponer arrays

```
// Dato: { nombre: "Jon", notas: [{asig: "ABD", nota: 7.5},
//                                {asig: "SI", nota: 8.0}] }
// $unwind crea un documento por cada elemento del array:

db.alumni.aggregate([
  { $unwind: "$notas" }
])
// Resultado:
// { nombre: "Jon", notas: { asig: "ABD", nota: 7.5 } }
// { nombre: "Jon", notas: { asig: "SI", nota: 8.0 } }

// Útil para después hacer $group por asignatura:
db.alumni.aggregate([
  { $unwind: "$notas" },
  { $group: {
      _id: "$notas.asignatura",
      media: { $avg: "$notas.nota" }
  }}
])
```

$lookup: unión entre colecciones

```
db.pedidos.aggregate([
   {
    $lookup: {
         from: "alumni",
         localField: "cliente",
         foreignField: "nombre",
         as: "alumniMatch"
     }
   }
])
```

Diseño del esquema en MongoDB

Aunque MongoDB es schemaless, el diseño del esquema es **fundamental**

La pregunta clave: ¿**embeber** o **referenciar**?

- **Embeber**: incluir los datos relacionados dentro del documento
- **Referenciar**: guardar el _id del documento relacionado

La elección depende de patrones de acceso, tamaño y frecuencia de actualización

Embeber (desnormalizar)

```
// Toda la información en un solo documento
{
  nombre: "Javier Cansado",
  direccion: {
    calle: "Gran Vía 1",
    ciudad: "Bilbao"
  },
  pedidos: [
    { producto: "Monitor", precio: 250, fecha: "2026-01-15" },
    { producto: "Ratón", precio: 25, fecha: "2026-02-20" }
  ]
}
```

Ventajas: lectura muy rápida (una sola consulta)

Inconvenientes: datos duplicados, documentos muy grandes

Referenciar (normalizar)

```
// Colección "clientes"
{
  _id: ObjectId("aaa..."),
  nombre: "Javier Cansado",
  direccion: { calle: "Gran Vía 1", ciudad: "Bilbao" }
}

// Colección "pedidos"
{
  cliente_id: ObjectId("aaa..."),
  producto: "Monitor",
  precio: 250,
  fecha: "2026-01-15"
}
```

Ventajas: sin duplicación, documentos más pequeños

Inconvenientes: requiere $lookup (más lento), múltiples consultas

¿Cuándo embeber? ¿Cuándo referenciar?

Criterio	Embeber	Referenciar
Relación	1:1, 1: pocos	1: muchos, N:M
Datos se leen juntos	Sí	No siempre
Datos cambian mucho	No	Sí
Tamaño subdatos	Pequeño/limitado	Grande/ilimitado
Prioridad	Velocidad lectura	Consistencia

Índices en MongoDB

Sin índices, MongoDB debe examinar **todos** los documentos (collection scan)

Un índice permite encontrar documentos sin escanear toda la colección

MongoDB crea automáticamente un índice sobre `_id`

Se pueden crear índices sobre cualquier campo o combinación de campos

Crear y gestionar índices

```
// Cargar ehu.alumni.json en coleccion ehu.alumni1000

// Ver índices existentes:
db.alumni1000.getIndexes()

// Crear un índice simple:
db.alumni1000.createIndex({ email: 1 }) // 1 = ascendente; -1 = descendente

// Índice compuesto (sobre múltiples campos):
db.alumni1000.createIndex({ nombre: 1, edad: -1 })

// Índice único (no permite duplicados):
db.alumni1000.createIndex({ email: 1 }, { unique: true })

// Eliminar un índice:
db.alumni1000.dropIndex({ email: 1 })
```

explain(): analizar consultas

```
db.alumni1000.find({ email: "alumno0680@ikasle.ehu.eus" }).explain("executionStats")
// COLLSCAN: sin índice, escanea toda la colección
// IXSCAN: con índice, busca directamente
```

Crear índices en campos que se usan frecuentemente en consultas

Los índices aceleran las lecturas pero ralentizan las escrituras

Validación de esquema

Aunque MongoDB es schemaless, se puede definir validación opcional

Usar JSON Schema para definir reglas sobre los documentos

```
db.createCollection("ehu-schema", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nombre", "email", "edad"],
      properties: {
        nombre: { bsonType: "string",
                  description: "debe ser un string" },
        email:  { bsonType: "string",
                  pattern: "^.+@.+$" },
        edad:   { bsonType: "int",
                  minimum: 16, maximum: 99 }
      }
    }
  }
})
```

Validación: niveles

Nivel "strict" (por defecto): valida en insert y update

Nivel "moderate": valida solo docs nuevos o válidos existentes

Permite equilibrar flexibilidad de esquema e integridad de datos

collMod_moderate, collMod_strict, productos_ejemplo, invalid_productos

Buenas prácticas con MongoDB

- Diseñar documentos según los patrones de consulta más frecuentes
- Elegir entre embeber y referenciar con criterios de acceso y tamaño
- Crear índices en campos usados en filtros y ordenaciones
- Aplicar validación de esquema cuando la integridad sea crítica
- Monitorizar consultas con `explain()` y métricas de rendimiento

MongoDB / Triple Stores

¿Se puede cargar un archivo RDF (JSON-LD) en MongoDB?

¿GraphDB / Ontop?